

Block I: Databases, Data modeling & Storage

1. What is a Database?

A database is an organized collection of related data stored in a structured manner so that it can be easily accessed, managed, and updated.

2. What is DBMS?

A Database Management System (DBMS) is software that allows users to create, manage, and manipulate databases.

Examples: MySQL, Oracle, SQL Server.

3. What are the Advantages of DBMS?

- Reduces data redundancy
 - Improves data consistency
 - Provides data security
 - Allows data sharing
 - Backup and recovery
-

4. What are the Disadvantages of DBMS?

- High cost
 - Complexity
 - Requires skilled personnel
 - Large storage requirement
-

5. What is Data Redundancy?

Data redundancy is the unnecessary duplication of data in a database.

6. What is Data Independence?

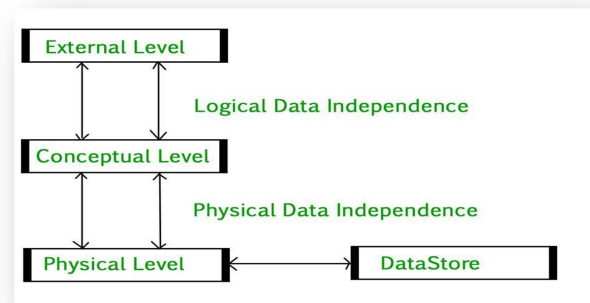
Data independence means changes in database structure do not affect application programs.

Types:

- Logical data independence
 - Physical data independence
-

7. Explain the Three-Level Architecture of DBMS.

1. External Level – User view
2. Conceptual Level – Logical structure
3. Internal Level – Physical storage



This architecture provides data independence.

i. External Level (View Level)

*Image Source: Geeksforgeeks
(<https://www.geeksforgeeks.org/dbms/introduction-of-3-tier-architecture-in-dbms-set-2/>)
*Used for educational purposes

This is the topmost level of the architecture.

It describes how different users view the database.

Each user can have a different view according to their needs.

For example:

- A student can see only marks and name.
- An admin can see complete details.

Features:

- Provides customized views
- Enhances security
- Hides unnecessary data

It is also called **User View Level**.

ii. Conceptual Level (Logical Level)

This is the middle level of architecture.

It describes the overall logical structure of the entire database.

It defines:

- Entities
- Attributes
- Relationships
- Constraints

At this level:

- Physical storage details are hidden.
- The entire database structure is defined.

It acts as a bridge between external and internal levels.

iii. Internal Level (Physical Level)

This is the lowest level of architecture.

It describes how data is physically stored in memory.

It includes:

- File organization
- Indexing
- Data storage techniques
- Access methods

Users do not interact directly with this level.

Advantages of Three-Level Architecture

- Provides data abstraction
- Improves security

- Supports data independence
- Easy database modification
- Multiple user views supported

8. What is a Data Model?

A data model is a collection of concepts used to describe the structure of a database.

Examples:

- Hierarchical model
- Network model
- Relational model

9. What is Schema and Instance?

Schema: Structure of database (design).

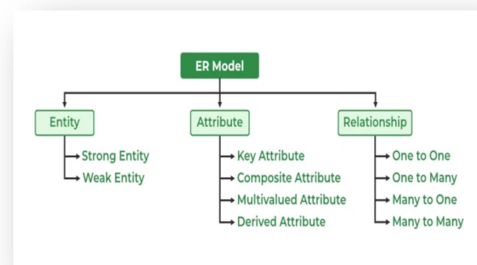
Instance: Actual data stored at a particular time.

10. What is E-R Model?

The **ER Model (Entity–Relationship Model)** is a high-level conceptual data model used to design the structure of a database.

It was proposed by Peter Chen in 1976.

The ER model is mainly used during the **database design phase** to represent real-world objects and their relationships in a simple graphical form called an **ER Diagram (ERD)**.



***Image Source: Geeksforgeeks**
(<https://www.geeksforgeeks.org/dbms/introduction-of-er-model/>) *Used for educational purposes

❖ Purpose of ER Model

- To design database structure visually
- To represent real-world entities
- To show relationships between entities
- To simplify database planning

It acts as a blueprint before creating tables in relational database.

❖ Main Components of ER Model

The ER Model has three main components:

i. Entity

An Entity is a real-world object or concept that can be uniquely identified.

Examples:

- Student
- Teacher
- Course

In ER Diagram, entity is represented by a rectangle.

Example: [Student]

ii. Attribute

Attributes describe the properties of an entity.

Example:

Student entity may have:

- RollNo
- Name
- Age

Types of Attributes:

- Simple Attribute (Age)
- Composite Attribute (FullName → FirstName + LastName)
- Multivalued Attribute (PhoneNumbers)
- Derived Attribute (Age derived from DOB)

Attributes are represented by oval shapes in ER diagram.

iii. Relationship

A Relationship shows how two or more entities are related.

Example:

Student “enrolls in” Course

Relationship is represented by a diamond shape.

Example:

Student ◇ Enrolls ◇ Course

❖ Types of Relationships

1. One-to-One (1:1)
2. One-to-Many (1:M)
3. Many-to-Many (M:N)

Example: One teacher teaches many students → 1:M

❖ ER Diagram Symbols

- Rectangle → Entity
- Oval → Attribute
- Diamond → Relationship
- Line → Connection

(Drawing small ER diagram in exam gives extra marks □)

❖ Advantages of ER Model

- Easy to understand
- Visual representation
- Helps in database design
- Reduces design errors

- Improves communication between developer and client
-

11. What is an Entity?

An entity is a real-world object that can be identified uniquely.

Example: Student, Employee.

12. What is an Attribute?

An attribute is a property of an entity.

Example: Student_ID, Name.

13. What is a Relationship?

A relationship represents an association between two or more entities.

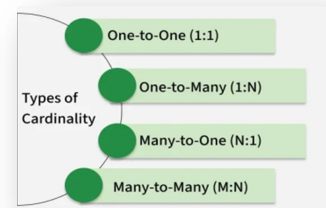
Example: Student "enrolls in" Course.

14. What is Cardinality?

Cardinality defines the number of entities that can participate in a relationship.

Types:

- One-to-One
- One-to-Many
- Many-to-Many



*Image Source: Geeksforgeeks
(<https://www.geeksforgeeks.org/dbms/cardinality-in-dbms/>)
*Used for educational purposes

15. What is Primary Key?

A primary key is an attribute that uniquely identifies each record in a table.

16. What is Secondary Storage?

Secondary storage refers to non-volatile storage devices used to store large amounts of data permanently.

Example: Hard disk.

17. What is File Organization?

File organization refers to the way records are physically arranged on storage devices.

Types:

- Heap file
 - Sequential file
 - Indexed file
 - Hashed file
-

18. What is Hashing?

Hashing is a technique used to quickly access data by using a hash function to compute the location of a record.

19. What is Indexing?

Indexing is a technique used to improve the speed of data retrieval.

Types:

- Single-level index
- Multi-level index
- B-tree
- B+ tree

20. Differentiate Between B-Tree and B+ Tree.

B-Tree	B+ Tree
• Data stored in all nodes	• Data stored only in leaf nodes
• Slower sequential access	• Faster sequential access
• Less commonly used	• Widely used in DBMS

Block II: Data Models & Concepts

1. Explain the Relational Data Model.

The Relational Data Model was proposed by E.F. Codd in 1970. In this model, data is stored in the form of tables called relations. Each relation consists of rows (tuples) and columns (attributes).

A relation has:

- Relation name
- Attributes (columns)
- Tuples (rows)
- Domain (set of valid values)

Each relation must have a primary key to uniquely identify records.

Advantages:

- Simple structure
- Data independence
- Reduced redundancy
- Strong integrity rules

It is the most widely used model in DBMS like MySQL and Oracle.

2. Explain Basic Concepts of Relational Model.

The relational model is based on the concept of relation (table).

Important concepts:

- Relation – Table
- Tuple – Row
- Attribute – Column
- Domain – Set of values
- Degree – Number of attributes
- Cardinality – Number of tuples

Relations follow properties:

- No duplicate tuples
- Order of rows/columns does not matter
- Each cell contains atomic value

3. Explain Keys in Relational Database.

Keys are attributes that uniquely identify tuples.

Types:

- Super Key – Set of attributes uniquely identifying tuple
- Candidate Key – Minimal super key
- Primary Key – Selected candidate key
- Alternate Key – Other candidate keys
- Foreign Key – Refers to primary key of another table
- Composite Key – More than one attribute

Keys maintain uniqueness and relationship between tables.

4. Explain Relational Model Constraints.

Constraints maintain accuracy and consistency.

Types:

1. Domain Constraint – Valid data type
2. Entity Integrity – Primary key cannot be NULL
3. Referential Integrity – Foreign key must match primary key
4. Key Constraint – No duplicate primary key values

Constraints prevent invalid and inconsistent data.

5. Explain Update Operations in Relational Model.

There are three operations:

1. Insert – Adds new tuple (may violate constraints)
2. Delete – Removes tuple (may affect referential integrity)
3. Update – Modifies existing data

These operations must follow integrity constraints.

6. Explain Relational Algebra.

Relational Algebra is a procedural query language used to manipulate relations.

It performs operations on tables and produces new relations as output.

It forms the theoretical foundation of SQL.

Two categories:

- Unary Operations
 - Binary Operations.
-

7. Explain Selection and Projection Operations.

Selection (σ): Selects rows based on condition.

Example: $\sigma \text{ Age} > 20$ (Student)

Projection (π): Selects specific columns.

Example: $\pi \text{ Name, Age}$ (Student)

Selection works on rows.
Projection works on columns.

8. Explain Set Operations in Relational Algebra.

Set operations include:

- Union ($\cup$) – Combines tuples of two relations
- Intersection ($\cap$) – Common tuples
- Difference ($-$) – Tuples in one relation but not in another

Conditions:

Relations must be union-compatible (same attributes and domain).

9. Explain Cartesian Product and Join Operation.

In Relational Algebra, Cartesian Product and Join are important binary operations. Both operations combine two relations (tables) to produce a new relation.

1. Cartesian Product ($\times$)

Cartesian Product is a binary operation that combines every tuple of one relation with every tuple of another relation.

Symbol: $\times$ **General Form:**

$$R \times S$$

Where R and S are two relations.

Definition:

If relation R has m tuples and relation S has n tuples, then the result of $R \times S$ will contain $m \times n$ tuples.

Example:

Suppose we have two tables:

R (Student)

Roll	Name
1	Amit
2	Rahul

- Diagram (Recommended in Exam)

S (Course)

CourseID	CourseName
C1	BCA
C2	BBA

$R \times S$ will produce:

Roll	Name	CourseID	CourseName
1	Amit	C1	BCA
1	Amit	C2	BBA
2	Rahul	C1	BCA
2	Rahul	C2	BBA

Total rows = $2 \times 2 = 4$

Characteristics:

- Combines all possible pairs of tuples
 - Degree = Degree of R + Degree of S
 - Produces large number of rows
 - Usually used before applying join condition
-

2. Join Operation ($\bowtie$)

Join operation is used to combine related tuples from two relations based on a condition.

Join is basically a Cartesian product followed by a selection condition.

General Form:

$R \bowtie \text{condition } S$

Types of Join:

1. Theta Join (θ -Join)

Uses any comparison operator ($=, <, >, \leq, \geq$)

Example:

$R \bowtie R.Roll = S.Roll \ S$

2. Equi Join

Special case of theta join using only equality ($=$).

3. Natural Join

Automatically joins two relations based on common attributes and removes duplicate columns.

Example:

$Student \bowtie Course$

Example of Join:

Student (Roll, Name, CourseID)

Course (CourseID, CourseName)

$Student \bowtie Student.CourseID = Course.CourseID \ Course$

Result will contain only matching rows.

➤ Difference Between Cartesian Product and Join

Cartesian Product	Join
Combines all tuples	Combines only related tuples
No condition	Condition required
Produces large result	Produces meaningful result
Rarely used alone	Frequently used

10. Explain SQL and its Classification.

Introduction to SQL

SQL stands for **Structured Query Language**.

It is a standard language used to create, manage, and manipulate relational databases.

SQL is used in relational database management systems (RDBMS) like MySQL, Oracle Database, and Microsoft SQL Server.

SQL allows users to:

- Create database structures
 - Insert and retrieve data
 - Update and delete records
 - Control access permissions
 - Manage transactions
-

➤ Classification of SQL

SQL commands are divided into four main categories:

- i. Data Definition Language (DDL)
 - ii. Data Manipulation Language (DML)
 - iii. Data Control Language (DCL)
 - iv. Transaction Control Language (TCL)
-

3. Data Definition Language (DDL)

DDL is used to define and modify the structure of database objects such as tables, views, and indexes.

Common DDL Commands:

- **CREATE** – Creates new table or database
Example: `CREATE TABLE Student (Roll INT, Name VARCHAR(20));`
 - **ALTER** – Modifies table structure
 - **DROP** – Deletes table
 - **TRUNCATE** – Removes all records from table
- ❖ DDL commands automatically save changes (auto-commit).
-

4. Data Manipulation Language (DML)

DML is used to manipulate data stored in tables.

Common DML Commands:

- **INSERT** – Adds new records
- **UPDATE** – Modifies existing records
- **DELETE** – Removes records
- **SELECT** – Retrieves data

Example: `SELECT * FROM Student;`

- ❖ DML commands can be rolled back using TCL commands.
-

5. Data Control Language (DCL)

DCL is used to control access and permissions in a database.

Common DCL Commands:

- **GRANT** – Gives privileges to users
- **REVOKE** – Removes privileges

Example: `GRANT SELECT ON Student TO User1;`

- ❖ Ensures database security.
-

6. Transaction Control Language (TCL)

TCL is used to manage transactions in the database.

Common TCL Commands:

- **COMMIT** – Saves changes permanently
- **ROLLBACK** – Undoes changes
- **SAVEPOINT** – Creates rollback point

These commands help maintain data consistency and reliability.

Summary Table

Category	Purpose
DDL	Defines database structure
DML	Manipulates data
DCL	Controls access
TCL	Manages transactions

11. Explain DDL Commands with Examples.

DDL defines structure of database.

Commands:

CREATE – Creates table

ALTER – Modifies table

DROP – Deletes table

TRUNCATE – Removes all records

Example:

```
CREATE TABLE Student (Roll INT, Name VARCHAR(20));
```

12. Explain DML Commands with Examples.

DML manipulates data.

INSERT – Adds data

UPDATE – Modifies data

DELETE – Removes data

SELECT – Retrieves data

Example:

INSERT INTO Student VALUES (1, 'Rahul');

13. Explain Hierarchical Data Model.

In hierarchical model, data is organized in tree structure.

Features:

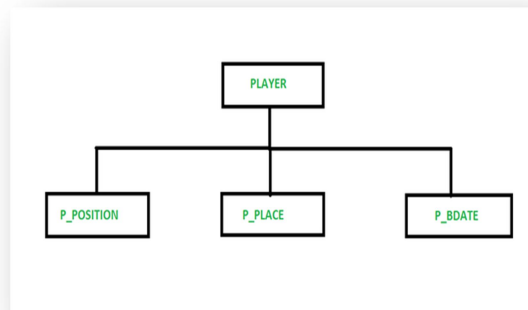
- One-to-many relationship
- Each child has only one parent

Advantages:

- Simple structure

Disadvantages:

- Cannot handle many-to-many



*Image Source: Geeksforgeeks

(<https://www.geeksforgeeks.org/dbms/hierarchical-model-in-dbms/>)

*Used for educational purposes

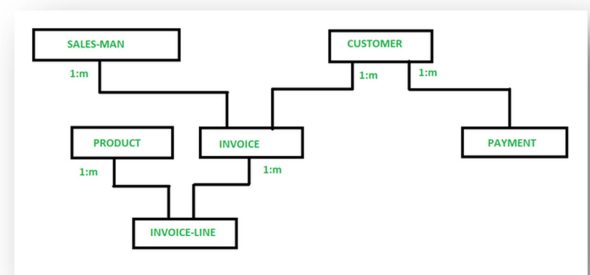
14. Explain Network Data Model.

In network model, data is organized in graph structure.

Features:

- Many-to-many relationships
- Uses pointers

Advantages:



*Image Source: Geeksforgeeks

(<https://www.geeksforgeeks.org/dbms/network-model-in-dbms/>)

*Used for educational purposes

- Flexible

Disadvantages:

- Complex structure
-

15. Compare Hierarchical, Network and Relational Models.

Data models define how data is organized, stored, and related in a database. The three important data models are:

1. Hierarchical Model
 2. Network Model
 3. Relational Model
-

1. Hierarchical Model

The Hierarchical Model organizes data in a tree-like structure.

- Follows parent–child relationship
- One parent can have many children
- Each child has only one parent
- Supports one-to-many relationship

It was widely used in early database systems like IBM Information Management System.

2. Network Model

The Network Model organizes data in a graph structure.

- Supports many-to-many relationships
- A child can have multiple parents
- Uses pointers to connect records

It is more flexible than the hierarchical model but more complex to design.

3. Relational Model

The Relational Model organizes data in tables (relations).

- Data stored in rows and columns
- Relationships maintained using primary and foreign keys
- Supports one-to-one, one-to-many, and many-to-many relationships
- Based on the theory proposed by Edgar F. Codd

It is the most widely used model in modern DBMS.

➤ Comparison Table

Feature	Hierarchical Model	Network Model	Relational Model
Structure	Tree	Graph	Table
Relationship	One-to-Many	Many-to-Many	All types supported
Flexibility	Low	Moderate	High
Complexity	Simple	Complex	Simple & User-friendly
Data Access	Navigational	Navigational	Non-navigational (SQL)
Data Independence	Low	Low	High
Redundancy	High	Moderate	Low
Example DBMS	IMS	IDMS	MySQL, Oracle

.

16. Explain Concept of Domain and Degree.

In the relational data model, every relation (table) consists of attributes (columns) and tuples (rows). Two important concepts related to attributes are **Domain** and **Degree**.

i.Domain

A **Domain** is the set of all possible valid values that an attribute can take.

In simple words, domain defines the data type and permissible values for a column.

Example:

Student (RollNo, Name, Age)

- RollNo → Domain = Positive integers
- Name → Domain = Characters (String)

- Age $\rightarrow$ Domain = Integers between 0–100

So, each attribute must take values only from its defined domain.

Importance of Domain:

- Prevents invalid data entry
- Maintains data consistency
- Enforces domain constraint
- Improves data integrity

For example, if Age = "ABC", it violates the domain constraint.

ii. Degree

Degree of a relation is the number of attributes (columns) present in a relation.

Formula:

Degree = Number of attributes in a table

Example:

Student (RollNo, Name, Age, Course)

Here, total attributes = 4

So, Degree = 4

If a relation has:

- 1 attribute $\rightarrow$ Unary relation
- 2 attributes $\rightarrow$ Binary relation
- 3 attributes $\rightarrow$ Ternary relation
- n attributes $\rightarrow$ n-ary relation

Difference between Domain and Degree

Domain	Degree
Set of valid values for an attribute	Number of attributes in a relation
Related to column values	Related to table structure
Ensures data validity	Indicates table width

17. Explain Entity and Referential Integrity.

Entity Integrity: Primary key cannot be NULL.

Referential Integrity: Foreign key must match primary key or be NULL.

These rules maintain consistency.

18. Explain Conventional Data Models.

Conventional Data Models are the early database models that were used before the relational data model became popular. These models focus more on physical storage and data structure rather than logical representation.

The two main conventional data models are:

1. Hierarchical Data Model
2. Network Data Model

i. Hierarchical Data Model

In the hierarchical model, data is organized in a tree-like structure. It follows a parent–child relationship.

Features:

- Data is arranged in levels.
- One parent can have multiple children.
- Each child has only one parent.
- Supports one-to-many relationship.

Example:

A company database where one manager supervises many employees.

Advantages:

- Simple structure
- Fast data retrieval for hierarchical relationships

Disadvantages:

- Cannot support many-to-many relationships
- Difficult to modify structure
- Data redundancy possible

📌 □ Diagram Recommended:

Draw a tree structure showing Parent → Child nodes.

ii. Network Data Model

The network model is an extension of the hierarchical model. It organizes data in the form of a graph.

Features:

- Supports many-to-many relationships
- A child can have multiple parents
- Uses pointers to connect records

Example:

Student can enroll in multiple courses, and a course can have multiple students.

Advantages:

- More flexible than hierarchical model
- Efficient for complex relationships

Disadvantages:

- Complex structure
- Difficult to design and maintain
- Requires technical knowledge

Comparison with Relational Model

Conventional models use physical pointers and complex structures.

The relational model uses tables and logical relationships through keys, making it simpler and more flexible.

19. Explain Relational Database Languages in Detail.

Relational Database Languages are special languages used to define, manipulate, control, and manage data in a relational database system. These languages are mainly implemented through SQL (Structured Query Language).

Relational database languages are divided into four main categories:

i. Data Definition Language (DDL)

DDL is used to define and modify the structure of database objects such as tables, schemas, indexes, etc.

Common DDL Commands:

- **CREATE** – Creates new database objects
Example:
`CREATE TABLE Student (Roll INT, Name VARCHAR(20));`
- **ALTER** – Modifies existing table structure
Example:
`ALTER TABLE Student ADD Age INT;`
- **DROP** – Deletes table or database
Example:
`DROP TABLE Student;`
- **TRUNCATE** – Removes all records from a table

DDL commands automatically commit changes in most DBMS.

ii. Data Manipulation Language (DML)

DML is used to manipulate data stored in the database.

Common DML Commands:

- **INSERT** – Adds new records
- **UPDATE** – Modifies existing records
- **DELETE** – Removes records
- **SELECT** – Retrieves data

Example:

```
INSERT INTO Student VALUES (1, 'Rahul');  
SELECT * FROM Student;
```

DML commands can be rolled back using TCL commands.

iii. Data Control Language (DCL)

DCL is used to control access and permissions in a database.

Common DCL Commands:

- **GRANT** – Gives privileges to users
- **REVOKE** – Removes privileges

Example:

```
GRANT SELECT ON Student TO User1;
```

DCL ensures database security.

iv. Transaction Control Language (TCL)

TCL is used to manage transactions in a database.

Common TCL Commands:

- **COMMIT** – Saves changes permanently
- **ROLLBACK** – Undoes changes
- **SAVEPOINT** – Creates rollback point

These commands help maintain consistency and reliability of data.

Conclusion

Relational database languages provide complete control over database systems.

- DDL defines structure
- DML manipulates data
- DCL controls access
- TCL manages transactions

Together, they make SQL a powerful language for managing relational databases.

20. Explain Advantages and Limitations of Relational Model.

The **Relational Model** was proposed by Edgar F. Codd in 1970.

In this model, data is stored in the form of tables (relations) consisting of rows and columns.

It is the most widely used data model in modern database systems.

Advantages of Relational Model

1. Simple Structure

Data is organized in tables (rows and columns), which makes it easy to understand and use.

Even non-technical users can understand table format easily.

2. Data Independence

The relational model supports:

- Logical Data Independence
- Physical Data Independence

Changes in physical storage do not affect application programs.

3. Reduced Data Redundancy

Normalization techniques help in reducing duplicate data.

This minimizes storage waste and avoids inconsistency.

4. Strong Integrity Constraints

Relational model supports:

- Domain Constraint
- Entity Integrity
- Referential Integrity

These constraints ensure accuracy and consistency of data.

5. Easy Data Retrieval

Using SQL (Structured Query Language), data can be retrieved easily with simple queries.

Complex operations can be performed using JOIN, GROUP BY, etc.

6. Security

Access control can be applied using GRANT and REVOKE commands.

Different users can have different access rights.

7. Flexibility

Tables can be modified using ALTER command without affecting the entire database structure.

❖ Limitations of Relational Model

1. Complex Joins Reduce Performance

When many tables are joined together, performance may decrease.

Large databases with complex queries can become slow.

2. Not Suitable for Hierarchical Data

Hierarchical or graph-like data is not handled efficiently compared to NoSQL models.

3. Limited Handling of Multimedia Data

Relational databases are not ideal for storing large multimedia objects like videos or images.

4. Fixed Schema

Relational model requires predefined schema.

Changing schema frequently can be difficult in large systems.

5. Scalability Issues

Horizontal scaling (distributing data across many servers) is more difficult compared to modern NoSQL databases.

❖ Summary Table

Advantages	Limitations
Simple structure	Complex joins affect speed
Data independence	Not ideal for hierarchical data
Reduced redundancy	Fixed schema
Strong constraints	Limited multimedia handling
Security support	Scalability challenges

Block III: Database Design & Transaction Concept

1. What is Database Design?

Definition:

Database design is the process of creating a detailed data model for a database. It involves organizing data, defining relationships, attributes, and constraints to ensure efficiency, consistency, and integrity.

Steps in Database Design:

1. Requirements Analysis – Collect user and business requirements
2. Conceptual Design – Create ER diagrams
3. Logical Design – Convert ER diagram to relational tables, define keys
4. Physical Design – Decide storage structures, indexes, and optimization
5. Implementation – Create actual database in DBMS

Example: A university database – entities: Student, Course, Department; relationships: Student enrolls in Course

Advantages:

- Reduces redundancy
- Ensures data integrity
- Improves query efficiency

2. Explain Conceptual, Logical, and Physical Design

1. Conceptual Design:

- High-level design using ER model
- Focus on what data to store
- Independent of DBMS
- Output: ER diagram

2. Logical Design:

- Converts ER diagram into relational tables
- Defines primary keys, foreign keys, constraints
- Still DBMS-independent

3. Physical Design:

- Defines how data is stored on disk
- File organization, indexes, and access paths
- DBMS-specific

Diagram (Recommended in Exam):

```
User View (External)
      ↓
Logical Structure (Conceptual)
      ↓
Physical Storage (Internal)
```

3. What is Normalization?

Definition:

Normalization is the process of organizing tables to reduce redundancy, avoid anomalies, and ensure data integrity.

Goals:

- Eliminate duplicate data
- Prevent update, insert, delete anomalies
- Ensure functional dependencies

Example:

A table storing Student info with multiple courses → Split into Student and Course tables to avoid repeating courses.

Advantages:

- Data consistency
- Easier maintenance

Limitations:

- May increase number of tables → joins increase

4. Explain First Normal Form (1NF)

Definition:

A relation is in 1NF if all attributes have atomic values, and there are no repeating groups.

Example:

Before 1NF:

Student	Courses
Rahul	BCA, MCA
Amit	BBA

After 1NF:

Student	Course
Rahul	BCA
Rahul	MCA
Amit	BBA

Points:

- Degree may remain same
- Cardinality increases

Advantages:

- Ensures atomicity of data
 - Makes table compatible for relational operations
-

5. Explain Second Normal Form (2NF)

Definition:

A relation is in 2NF if:

1. It is in 1NF
2. Every non-prime attribute is fully functionally dependent on the **entire primary key**

Example:

Table: Enrollment (StudentID, CourseID, StudentName)

- Primary key = (StudentID, CourseID)
- StudentName depends only on StudentID → partial dependency

2NF Tables:

- Student (StudentID, StudentName)
- Enrollment (StudentID, CourseID)

Advantages:

- Eliminates partial dependency
 - Reduces redundancy
-

6. Explain Third Normal Form (3NF)

Definition:

A relation is in 3NF if:

1. It is in 2NF
2. No transitive dependency exists (non-prime attribute depends on another non-prime attribute)

Example:

Table: Student (StudentID, DepartmentID, DepartmentName)

- DepartmentName depends on DepartmentID, not StudentID
- Transitive dependency $\rightarrow$ violates 3NF

3NF Tables:

- Student (StudentID, DepartmentID)
- Department (DepartmentID, DepartmentName)

Advantages:

- Eliminates anomalies
 - Ensures all attributes depend on primary key
-

7. Explain Boyce-Codd Normal Form (BCNF)

Definition:

A table is in BCNF if it is in 3NF and **every determinant is a candidate key**.

Example:

Table: CourseInstructor (CourseID, Instructor, Department)

- Department $\rightarrow$ Instructor, violates BCNF
- Decompose into:
 - CourseInstructor (CourseID, Instructor)
 - DepartmentInstructor (Department, Instructor)

Advantages:

- Eliminates remaining anomalies
 - Stronger than 3NF
-

8. Explain Strong and Weak Entity

- **Strong Entity:** Can exist independently; has primary key
- **Weak Entity:** Cannot exist without owner entity; identified using partial key + foreign key

Example:

- Strong: Student
- Weak: Dependent of Employee (identified using EmployeeID)

ER Diagram:

- Weak entity → double rectangle
 - Partial key → dashed oval
-

9. Explain Relationship Types in ER Model

1. **One-to-One (1:1):** Each entity maps to one entity
2. **One-to-Many (1:M):** One entity maps to many entities
3. **Many-to-Many (M:N):** Many entities map to many entities

Example:

- 1:1 – Passport and Person
- 1:M – Teacher and Students
- M:N – Students and Courses

Diagram Recommended in Exam ☒

10. Explain Entity Integrity and Referential Integrity

- **Entity Integrity:** Primary key cannot be NULL
- **Referential Integrity:** Foreign key must reference a valid primary key or be NULL

Ensures **consistency and correctness** in relational tables.

11. Explain Transactions in DBMS

Definition:

A transaction is a sequence of operations treated as a single logical unit.

Properties (ACID):

- Atomicity
- Consistency
- Isolation

- Durability

Example: Bank transfer – debit from one account, credit to another

12. Explain ACID Properties

1. **Atomicity:** All operations succeed or none
2. **Consistency:** Database remains in valid state
3. **Isolation:** Concurrent transactions do not interfere
4. **Durability:** Changes persist even after system failure

Example:

Money transfer fails midway → rollback → ensures atomicity

13. Explain Concurrency Control

Ensures multiple transactions execute without conflict.

Techniques:

- Locking (shared/exclusive)
- Timestamp ordering
- Optimistic concurrency control

Purpose:

- Prevent lost updates
 - Prevent dirty reads
 - Maintain ACID properties
-

14. Explain Deadlock in DBMS

Definition: Two or more transactions wait indefinitely for each other's resources.

Example:

- T1 locks A, wants B
- T2 locks B, wants A

Solutions:

- Wait-Die / Wound-Wait schemes
 - Deadlock detection & recovery
-

15. Explain Commit and Rollback

- **COMMIT:** Saves transaction permanently
- **ROLLBACK:** Reverts transaction in case of error

Example: Transfer \$100 fails → ROLLBACK → accounts unchanged

16. Explain Savepoint in DBMS

Savepoint allows **partial rollback** within a transaction.

Example:

- Transaction has steps 1–5
- Savepoint after step 3
- Error in step 5 → rollback to savepoint 3

Advantages:

- Partial recovery without discarding whole transaction
-

17. Explain ER Diagram Symbols

Symbol	Meaning
Rectangle	Entity
Diamond	Relationship
Oval	Attribute
Double Oval	Multivalued Attribute
Double Rectangle	Weak Entity
Dashed Oval	Derived Attribute
Line	Connection

Diagram Recommended ☒

18. Explain Functional Dependency

Definition:

Attribute B is functionally dependent on attribute A ($A \rightarrow B$) if for each value of A there is exactly one value of B.

Example:

RollNo $\rightarrow$ Name

Importance:

- Used in normalization
- Ensures consistency

19. Explain Composite and Multivalued Attributes

- **Composite Attribute:** Can be divided into subparts (FullName $\rightarrow$ FirstName + LastName)
- **Multivalued Attribute:** Can have multiple values (PhoneNumbers)

Example:

- Student entity $\rightarrow$ FullName (composite), PhoneNumbers (multivalued)

Handled during table decomposition in normalization.

20. Explain Advantages and Limitations of Normalization

Advantages:

- Reduces redundancy
- Eliminates update/insert/delete anomalies
- Improves consistency

Limitations:

- More tables $\rightarrow$ complex joins
- Performance overhead for large DB
- Not always practical for reporting systems

